

Enunciado Evaluacion Parcial 1

Programacion Orientada a Objetos

FORMA C

Sigla	Asignatura	Experiencia de Aprendizaje
DSY1102	Programacion Orientada a Objetos	EA1: Fundamentos de Programacion Orientada a Objetos
Tiempo	Modalidad de Trabajo	Indicadores de logro
3 bloques	Individual	IL 1.1 al IL 1.6

Condiciones de la evaluacion

Del entorno y la entrega

- La evaluacion es de caracter **individual**. No esta permitida ninguna forma de colaboracion entre estudiantes.
- No se permite el acceso a internet ni el uso de telefonos, relojes inteligentes ni ningun otro dispositivo externo.
- El programa debe compilar y ejecutarse correctamente. Una solucion que no compile no permite evidenciar ningun indicador de logro.
- El proyecto debe entregarse comprimido en formato ZIP, con tu nombre completo como nombre del archivo, en la actividad habilitada en AVA antes de que finalice la sesion.
- No se aceptan entregas por ningun otro medio ni fuera del horario establecido. La ausencia de entrega en AVA genera automaticamente la nota minima.

De tu solucion

- El codigo debe organizarse en clases separadas, respetando las convenciones de nomenclatura de Java: `PascalCase` para nombres de clase, `camelCase` para metodos y atributos.

- Puedes crear metodos o clases auxiliares siempre que no contradigan los requerimientos del caso.
 - No se admite el uso de herramientas de inteligencia artificial ni de recursos digitales externos.
 - Cualquier indicio de colaboracion no autorizada, uso de IA o plagio implica la anulacion inmediata de la evaluacion con nota minima de 1.0.
-

Instrucciones de entrega con Git

Antes de comenzar a codificar, debes configurar Git y preparar tu repositorio. Sigue estos pasos en orden:

Paso 1 — Configurar tu identidad en Git (si no lo has hecho previamente)

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu.correo@duocuc.cl"
```

Paso 2 — Clonar el repositorio de tu seccion

Seccion 004D:

```
git clone https://github.com/hachimaki-dev/004D_Desarrollo_orientado_a_Objeto_2026
```

Seccion 002D:

```
git clone https://github.com/hachimaki-dev/002D_desarrollo_orientado_a_objeto_2026
```

Paso 3 — Ingresar al directorio del repositorio clonado

```
cd 004D_Desarrollo_orientado_a_Objeto_2026
```

Ajusta el nombre del directorio segun tu seccion.

Paso 4 — Crear una rama nueva para tu entrega

```
git checkout -b evaluacion1/Nombre_Apellido
```

Ejemplo: `git checkout -b evaluacion1/Carlos_Orellana`

Paso 5 — Desarrollar tu solucion dentro de esta rama

Crea tus archivos `.java` y trabaja tu solucion normalmente.

Paso 6 — Guardar tus cambios en un commit

```
git add .  
git commit -m "Entrega EP1 - Nombre Apellido - Confirmo que esta es mi entrega f
```

Paso 7 — Publicar tu rama en el repositorio remoto

```
git push origin evaluacion1/Nombre_Apellido
```

Importante: Además de publicar tu rama en Git, debes comprimir tu proyecto en formato ZIP (con tu nombre completo como nombre del archivo) y subirlo al AVA antes de que finalice la sesión. Ambas entregas son obligatorias.

Caso de negocio

Un taller mecánico necesita un sistema de consola en Java para gestionar las revisiones técnicas de vehículos. El taller atiende dos tipos de vehículos: **vehículos particulares** (automóviles de pasajeros) y **vehículos de carga** (camiones, furgones). Ambos comparten datos generales como patente, marca y número de revisiones pendientes, pero se diferencian en sus atributos específicos y en la forma de calcular el costo de cada revisión.

El sistema debe permitir registrar vehículos, listar los vehículos registrados, buscar vehículos por patente, realizar revisiones y ver un resumen general. Todas las entradas del usuario deben estar protegidas contra errores de ingreso.

Estructura de clases

El siguiente diagrama muestra la jerarquía de clases a implementar. La clase `Vehículo` es **abstracta** y define la estructura base. Las clases `VehículoParticular` y `VehículoCarga` heredan de ella y sobrescriben el cálculo del costo de revisión.

Vehiculo (abstracta)

```
- patente: String
- marca: String
- revisionesPendientes: int

+ Vehiculo(patente, marca, revisionesPendientes)
+ getPatente(): String
+ getMarca(): String
+ getRevisionesPendientes(): int
+ setRevisionesPendientes(int): void
+ calcularCostoRevision(): int
+ mostrarInfo(): String
```

▲ extends

VehiculoParticular

```
- numPasajeros: int

+ VehiculoParticular(patente, marca,
    revisionesPendientes, numPasajeros)
+ getNumPasajeros(): int
+ calcularCostoRevision(): int
+ mostrarInfo(): String
```

VehiculoCarga

```
- pesoMaxCarga: int

+ VehiculoCarga(patente, marca,
    revisionesPendientes, pesoMaxCarga)
+ getPesoMaxCarga(): int
+ calcularCostoRevision(): int
+ mostrarInfo(): String
```

Nota: Los costos se manejan como `int` (pesos chilenos enteros, sin decimales). El peso maximo de carga se expresa en kilogramos enteros. Las revisiones pendientes representan el numero de revisiones que el vehiculo tiene agendadas en el taller.

Requerimientos funcionales

A continuacion se detalla cada requerimiento. Las etiquetas **IL** indican que indicador de logro se evalua en cada parte.

R1 — Comentarios de paradigma **IL 1.1**

Al inicio de la clase `Main.java`, incluye un bloque de comentarios que explique:

- Que paradigma de programacion se esta usando y sus características principales.
- Al menos **2 diferencias concretas** entre el paradigma orientado a objetos (Java) y el paradigma que usabas antes (Python funcional). Por ejemplo: tipado, estructura del programa, compilacion vs. interpretacion, o forma de organizar el codigo.

R2 — Clase abstracta Vehículo **IL 1.3** **IL 1.4**

Implementa la clase abstracta `Vehículo` segun el diagrama UML:

- Atributos **privados**: `patente` (String), `marca` (String), `revisionesPendientes` (int).
- Constructor que reciba los 3 atributos.
- Getters para los tres atributos y setter para `revisionesPendientes`.
- Metodo abstracto `calcularCostoRevision()` que retorna `int`.
- Metodo `mostrarInfo()` que retorna un `String` con la patente, marca, costo de revision y revisiones pendientes. Este metodo puede ser sobrescrito por las clases hijas para agregar informacion adicional.

R3 — Clase VehículoParticular **IL 1.3** **IL 1.4**

Hereda de `Vehículo`. Representa un automovil de pasajeros.

- Atributo privado adicional: `numPasajeros` (int).
- Constructor que reciba los 4 parametros (3 del padre + `numPasajeros`). Usa `super()`.
- Getter para `numPasajeros`.
- `calcularCostoRevision()`: retorna `30000 + (numPasajeros * 2000)`.
- `mostrarInfo()`: sobrescribe (`@Override`) el metodo del padre. Agrega el tipo (`"Particular"`) y el numero de pasajeros al texto retornado.

R4 — Clase VehiculoCarga IL 1.3 IL 1.4

Hereda de `Vehiculo`. Representa un vehiculo de carga (camion, furgon).

- Atributo privado adicional: `pesoMaxCarga` (int, en kilogramos).
- Constructor que reciba los 4 parametros (3 del padre + `pesoMaxCarga`). Usa `super()`.
- Getter para `pesoMaxCarga`.
- `calcularCostoRevision()`: retorna `30000 + (pesoMaxCarga * 5)`.
- `mostrarInfo()`: sobrescribe (`@Override`) el metodo del padre. Agrega el tipo (`"Carga"`) y el peso maximo de carga al texto retornado.

R5 — Menu de consola con estructuras de control IL 1.2

La clase `Main` debe presentar un menu de consola que se repita hasta que el usuario elija salir. Usa un bucle `while` y `switch` para controlar las opciones:

```
≡≡≡ TALLER MECANICO ≡≡≡
1. Registrar vehiculo
2. Listar vehiculos
3. Buscar vehiculo por patente
4. Realizar revision
5. Resumen de vehiculos
6. Salir
Seleccione una opcion: _
```

Las opciones invalidas deben mostrar un mensaje y volver al menu sin que el programa se detenga.

R6 — Registrar vehiculo (con submenu) IL 1.2 IL 1.6

Al seleccionar "Registrar vehiculo", el sistema muestra un submenu para elegir entre Particular y Carga. Luego solicita los datos correspondientes y agrega el vehiculo a una coleccion `ArrayList<Vehiculo>`.

- Para vehiculos particulares, se solicita el numero de pasajeros como un valor entero positivo.
- Para vehiculos de carga, se solicita el peso maximo de carga en kilogramos como un valor entero positivo.
- Las revisiones pendientes deben ser un numero entero mayor o igual a 0.

R7 — Listar vehiculos (polimorfismo en accion)

IL 1.5

IL 1.6

Recorre la coleccion `ArrayList<Vehiculo>` e imprime la informacion de cada vehiculo usando `mostrarInfo()` .

Cada vehiculo se muestra con su formato propio (particular o carga) sin necesidad de hacer `instanceof` . Esto es polimorfismo: el mismo metodo produce resultados distintos segun el tipo real del objeto.

Si la lista esta vacia, mostrar un mensaje indicando que no hay vehiculos registrados.

R8 — Buscar vehiculo por patente

IL 1.2

IL 1.6

El usuario ingresa un texto y el sistema busca vehiculos cuya patente contenga ese texto (busqueda parcial, sin distinguir mayusculas/minusculas). Muestra todos los resultados encontrados o un mensaje si no hay coincidencias.

R9 — Realizar revision (descontar pendientes)

IL 1.2

IL 1.5

IL 1.6

El sistema lista los vehiculos con su indice. El usuario selecciona un vehiculo para realizar su revision. El sistema valida:

- Que el numero de vehiculo sea valido (exista en la lista).
- Que el vehiculo tenga al menos 1 revision pendiente.

Si la revision es valida, descuenta 1 unidad de `revisionesPendientes` y muestra un comprobante con la patente, la marca, el costo de la revision (calculado con `calcularCostoRevision()`) y las revisiones pendientes restantes.

R10 — Resumen de vehículos

IL 1.2

IL 1.5

IL 1.6

Muestra un resumen que incluya:

- Total de vehículos registrados.
- Cantidad de vehículos particulares y cantidad de vehículos de carga (usa `instanceof`).
- Ingresos proyectados: la suma de `calcularCostoRevision()` * `revisionesPendientes` de cada vehículo (lo que se facturaría si se completaran todas las revisiones pendientes).

R11 — Manejo de excepciones

IL 1.5

Toda entrada numérica del usuario debe estar protegida con `try-catch` . El programa **nunca debe detenerse** por un error de ingreso. Situaciones que deben manejarse:

- El usuario escribe texto cuando se espera un número (revisiones pendientes, número de pasajeros, peso de carga, índice de vehículo).
- El usuario ingresa un índice fuera de rango al realizar una revisión.
- El usuario intenta realizar una revisión en un vehículo con 0 revisiones pendientes.
- El usuario ingresa una opción de menú que no existe.

En cada caso, muestra un mensaje claro al usuario y vuelve al menú sin interrumpir la ejecución.

Ejemplo de ejecución esperada

Este ejemplo muestra como debería verse la interacción con el programa. Tu implementación debe producir una salida similar (no necesariamente idéntica en formato).

≡ TALLER MECANICO ≡

1. Registrar vehiculo
2. Listar vehiculos
3. Buscar vehiculo por patente
4. Realizar revision
5. Resumen de vehiculos
6. Salir

Seleccione una opcion: 1

--- Tipo de vehiculo ---

1. Particular
2. Carga

Seleccione: 1

Patente: ABCD12

Marca: Toyota

Revisiones pendientes: 2

Numero de pasajeros: 5

[OK] Vehiculo particular registrado.

Seleccione una opcion: 1

--- Tipo de vehiculo ---

1. Particular
2. Carga

Seleccione: 2

Patente: WXYZ99

Marca: Volvo

Revisiones pendientes: 1

Peso maximo de carga (kg): 3500

[OK] Vehiculo de carga registrado.

Seleccione una opcion: 2

--- VEHICULOS REGISTRADOS ---

[1] Particular | ABCD12 | Marca: Toyota | Pasajeros: 5 | Costo revision: \$40000

[2] Carga | WXYZ99 | Marca: Volvo | Carga max: 3500 kg | Costo revision: \$47500

Seleccione una opcion: 4

--- REALIZAR REVISION ---

[1] ABCD12 - Toyota (Pendientes: 2)

[2] WXYZ99 - Volvo (Pendientes: 1)

Ingrese numero de vehiculo: 1

Revision realizada.

Vehiculo: ABCD12 - Toyota

Costo de la revision: \$40000

Revisiones pendientes: 1

Seleccione una opcion: 5

--- RESUMEN DE VEHICULOS ---

Total de vehiculos: 2

Vehiculos particulares: 1

```
Vehiculos de carga: 1
Ingresos proyectados: $87500

Seleccione una opcion: 3

--- BUSCAR VEHICULO ---
Ingrese texto a buscar: AB
Resultados:
    Particular | ABCD12 | Marca: Toyota | Pasajeros: 5 | Costo revision: $40000 | 1

Seleccione una opcion: 4
Ingrese numero de vehiculo: abc
Error: Debe ingresar un numero valido.

Seleccione una opcion: 6
Programa finalizado.
```

Mapa de indicadores de logro

Esta tabla resume donde se evalua cada indicador. Usala como checklist antes de entregar.

IL	Indicador	Donde se evidencia
IL 1.1	Identificar el paradigma OO y diferenciarlo de otros	Comentarios en <code>Main.java</code> (R1)
IL 1.2	Estructuras de control y metodos con parametros y retorno	Menu while/switch (R5), submenu (R6), busqueda (R8), revision (R9)
IL 1.3	Clases con encapsulamiento y colaboracion	Atributos privados, getters/setters, constructores en <code>Vehiculo</code> , <code>VehiculoParticular</code> , <code>VehiculoCarga</code> (R2, R3, R4)
IL 1.4	Herencia, clases abstractas, jerarquias de tipos	Clase abstracta <code>Vehiculo</code> , herencia con <code>extends</code> , <code>super()</code> , metodo abstracto (R2, R3, R4)
IL 1.5	Polimorfismo y manejo de excepciones	Recorrido polimorfico del ArrayList (R7), <code>@Override</code> de metodos (R3, R4), try-catch (R11), instanceof en resumen (R10)
IL 1.6	Colecciones para almacenar y gestionar datos	<code>ArrayList<Vehiculo></code> como registro, agregar, iterar, buscar, modificar revisiones (R6, R7, R8, R9, R10)

Antes de entregar

Debe funcionar

El programa compila con `javac *.java` y se ejecuta con `java Main` sin errores.

Cada opcion del menu hace lo que se pide.

Los metodos sobrescritos (`@Override`) funcionan correctamente en cada subclase.

No debe romperse

Escribir "abc" cuando se pide un numero.

Intentar revisar un vehiculo con 0 revisiones pendientes.

Buscar un vehiculo cuando la lista esta vacia.

Seleccionar una opcion de menu inexistente.

Archivos a entregar

Comprime en formato **ZIP** todos los archivos `.java` de tu proyecto. Como minimo debes tener:

- `Main.java`
- `Vehiculo.java`
- `VehiculoParticular.java`
- `VehiculoCarga.java`

Importante: No se permite el uso de herramientas de IA, acceso a internet, ni dispositivos externos durante la evaluacion. Cualquier indicio de colaboracion no autorizada o uso de IA implica la anulacion de la evaluacion con nota minima 1.0.

Lee bien cada requerimiento, planifica antes de codificar y prueba tu programa antes de entregar.

